

Finishing I/O, and advanced I/O

Your program never talks to the disk, the network, or another program. It talks to the layer next to it. Every layer says *done* about something different [V50].

Ondřej (Ondra) Krajíček

me@ondrejkrasicek.com · www.linkedin.com/in/OndrejKrajicek

Built 2026-09-22 15:24 UTC

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

Where we stopped

Lecture 2 left four claims. Today asks each of them whose *done* that was [V50].

1. **Buffering pays for adjacent writes** [P1] — a seek forces the buffer out.
2. **A library call is not a system call** [P2] — `printf` shows up in `ltrace`, `write` in `strace` [V10].
3. **The boundary is observable** [P3] — a tracer prints the call and its return value [S1].
4. **Two kernels, two boundaries** [P4] — "the disagreement is about *which layer is nominated*" [V8].

Today adds one question to each: whose *done* was that? [V50]

[S1] `strace(1)` — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html>

Roadmap

Two blocks. The first finishes Lecture 2. The second is announced, and may run into Lecture 4.

- **Parts 1 to 5 · planned** — Lecture 2, finished from the slide it stopped on. Nothing is dropped.
- **Part 6 · planned** — so when is it safe? The question Lecture 2 left open.
- **Parts 7 to 9 · the announced tail** — waiting for input, three destinations, bytes you never touch.
- **Part 10 · planned** — voluntary homework, and the question Lecture 4 answers.

The tail is built in full. If the clock stops us, Lecture 4 opens on it.

Takeaways

- 1 Acknowledgement is not durability [T1]**

A call that reports success means the layer below accepted the bytes. The device keeps its own cache.
- 2 Readiness and completion [T2]**

Two different answers to one question: how does a program wait for I/O? *Tail*.
- 3 A file, a pipe and a socket [T3]**

One write carried a file, a pipe and a socket. *Tail*.
- 4 One call, many buffers [T4]**

The kernel can move bytes the program never touches. *Tail*.

PART 1

Recap by prediction

Three minutes, on paper, before I show you anything.

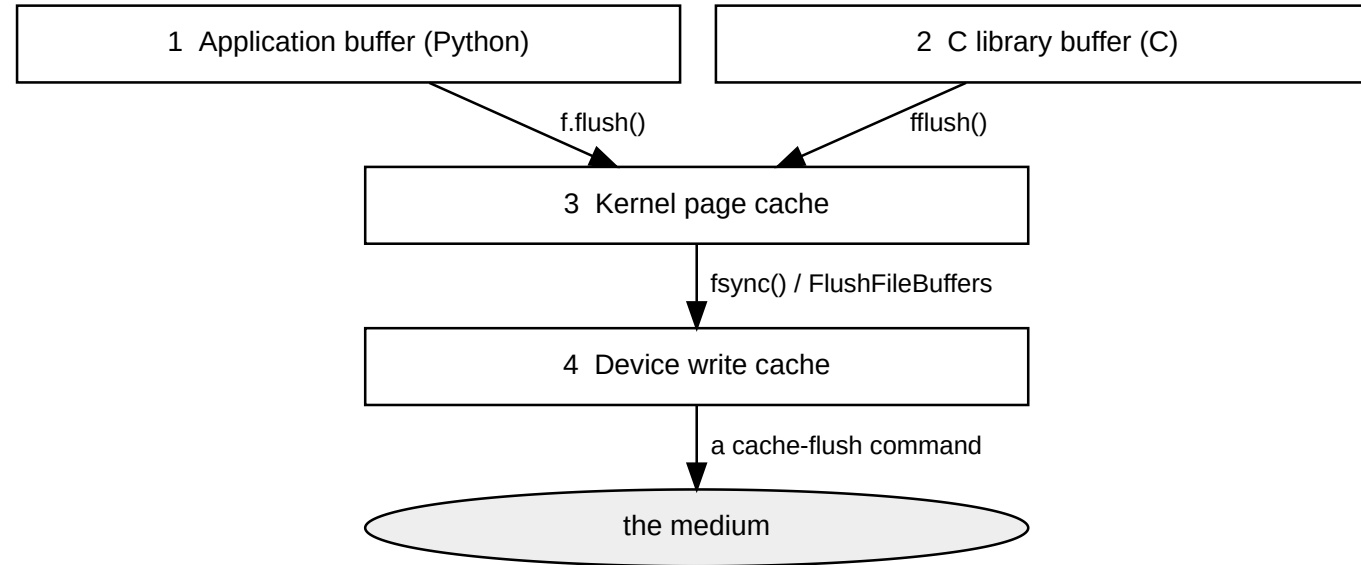
Predict: draw last week's stack

Three minutes. Nothing on the screen until you have written something down.

- 1** **Draw it** The four levels between your program and the medium. Name each one.
- 2** **Mark it** After `fflush()` returns, which level are the bytes in?
- 3** **Answer it** The machine loses power right now. What is lost?
Write all three. Nothing is handed in. I will ask several of you to read yours out [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

It is not one buffer — it is a stack



Levels 1 and 2 are alternative routes into level 3 [V6]. Each arrow is one call [S17] [S3] [S13] [V37] [V33].

[S17] `io.IOBase.flush()` — Python documentation. [S3] `fflush(3)` — Linux man-pages. [S13] `fsync` — The Open Group.

Explain: what dies with the process, what dies with the machine

Two different failures. Part 2's demonstration tests one. Your recap question asked about the other.

THE PROCESS DIES

Levels 1 and 2 go with it.
Anything already handed to the kernel survives, because the kernel writes it back on its own [V14].

THE MACHINE LOSES POWER

Levels 1, 2 and 3 are gone. Level 4 goes too, unless the device has power-loss protection.

Part 2 opens with `os._exit(0)`. That is the process dying, not the machine. [Part 6](#) measures the level-3 window.

Predict now, and keep it for an hour

The prediction for Part 6. You write it now, before Part 2, so you carry the question through the carried hour.

```
# demo/durability/save.py, the gist
with open("saved.bin", "wb") as f:      # default buffering; no flush, no fsync
    for _ in range(100):
        f.write(record)                # one 64-byte record
print("done")                          # the program's done. Whose?
```

The question

The process has exited. How long until the kernel actually writes your bytes to the device?

Your options

Under a second · about five seconds · about thirty seconds · not until shutdown.

One of four. Write it in the corner of your page. Do not change it later [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

What dies with the process

Parts 2 to 5 are Lecture 2, finished. Same slides, same order, renumbered into today.

Where Lecture 2 stopped

Last Tuesday ended after slide 32: the device's acknowledgement comes early [V20], and no tool in the room can check that report [V5].

From here to the end of Part 5 the slides are Lecture 2's, as built. The clock ran out after slide 32. Slide 33, the `os._exit` demonstration, is where we pick up.

Your two predictions from Part 1 stay on your page: the level the bytes are in after `fflush()`, and what a power cut takes. The first demonstration answers a different question — what a *process* dying takes — so keep both apart.

Three lines that print nothing

A demonstration. You have the stack now, so predict this one.

```
import os  
print("hello")  
os._exit(0)
```

To a terminal

Does hello appear?

Redirected to a file

What does the file contain?

Write down both answers before I run either one [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Why the file is empty

The rule sits in the C standard, not in the kernel.

- 1 Mode follows destination**

Output is "fully buffered if and only if the stream can be determined not to refer to an interactive device" [S5].
- 2 A terminal is interactive**

Bytes go out per line.
- 3 A file is not**

Bytes wait.
- 4 The exit skips cleanup**

No write reaches the kernel.

[S5] C99 draft §7.19.3. <https://busybox.net/~landley/c99-draft.html>

What this demonstration does not prove

It proves level one is real. It proves nothing about levels three and four.

"the honest classroom demo is a power-cut, not a kill -9. Killing the process loses only the *runtime's* buffer; the kernel still writes back." [V14]

- | | | |
|----------|-------------------------|---|
| 1 | What died | The process, and the level-one buffer with it. |
| 2 | What survived | Anything already in the page cache [V14]. |
| 3 | Wrong conclusion | "So flush() was enough. " It was enough against this test. |

Predict: what will `ltrace` show that `strace` did not?

Before I run the second tracer. Two lines on your page [S14].

It records the **library calls** a program makes [S31] — here, `fwrite` in the C library. `strace` sees `write` in the kernel instead.

- | | | |
|----------|--|---|
| 1 | The C program calls <code>fwrite</code> three times | How many <code>fwrite</code> lines does <code>ltrace</code> print? |
| 2 | The kernel saw one <code>write</code> | Does <code>ltrace -S</code> print that line too, or only the library calls? |

The answer follows from the stack you drew twenty minutes ago.

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/> [S31] `ltrace(1)`. <https://man7.org/linux/man-pages/man1/ltrace.1.html>

ltrace: the same idea, one layer up

strace stops at system calls. ltrace stops at calls into shared libraries.

What it does

It "intercepts and records the dynamic library calls" a process makes [S31]. It can also print "the system calls executed by the program" [S31].

Where it comes from

Juan Cespedes wrote it originally [S32]. The manual page names Petr Machata too [S31]. On Debian and Ubuntu you run `apt-get install ltrace` [S32].

HOW TO USE IT

```
ltrace ./iodemo      # the C library calls: fopen, fwrite, fclose
ltrace -S ./iodemo   # "system calls as well as library calls" [S31]
ltrace -c ./iodemo   # a count for each library call [S31]
```

It runs on Linux only.

[S31] ltrace(1). <https://man7.org/linux/man-pages/man1/ltrace.1.html> [S32] ltrace. <https://ltrace.org/>

Which levels can you actually watch?

I promised three of four boundaries. Here is the tool each needs.

BOUNDARY	WHAT WATCHES IT	IN THIS COURSE
App → library	ltrace sees it, strace does not [V10]	the tool above
Library → kernel	strace — where our tracer attaches [S1]	today
Kernel → device	a tracer for the kernel's block layer [V30]	elsewhere
Device → medium	no system-call trace shows whether the device flushed [V5]	no tracer

The same write, at three levels

Now that the levels have a reason to exist, walk down them with one small program.

The same write, at three levels

LEVEL	PYTHON	C	BUFFERED?
Application	<code>f.write(b"x")</code> [S6]	—	yes [S6]
C library	—	<code>fwrite(...)</code> [S3]	yes [S3]
System call	<code>os.write(...)</code> [S20]	<code>write(...)</code> [S2]	no

The bottom row takes a **file descriptor** — the small integer naming an open file. You saw one already: the `= 3` in the trace line was a file descriptor [S1].

[S1] `strace(1)` — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html> [S2] `write(2)` — Linux man-pages. <https://man7.org/linux/man-pages/man2/write.2.html> [S3] `fflush(3)` — Linux man-pages. [S6] Python documentation, `open()`. [S20] `os` — Miscellaneous operating system interfaces, `os.write()`. <https://docs.python.org/3/library/os.html#os.write>

Meet the program: `iodemo.c`

Nine lines of C. The first open fails on purpose, so the trace shows an error path.

```
f = fopen("no-such-file.txt", "rb"); /* 1. deliberate failure */
if (f != NULL) { fclose(f); return 2; }

f = fopen(path, "wb"); /* 2. the real file */
if (f == NULL) { return 1; }

fwrite("one\n", 1, 4, f); /* 3. three writes */
fwrite("two\n", 1, 4, f); /*    fourteen bytes */
fwrite("three\n", 1, 6, f);

if (fclose(f) != 0) { return 3; } /* 4. flush, then close */
```

It prints nothing, on purpose. A program that prints nothing adds no write lines of its own to the trace.

Three calls in the source. One in the trace.

3

`fwrite` calls in the source

14

bytes written in total

1

`write` call in the trace

The three `fwrite` calls go into one C library buffer. The manual calls that level "the user-space buffers provided by the C library" [S3]. The line to look for in the live trace, predicted from the source:

```
write(3, "one\ntwo\nthree\n", 14) = 14
```

The 3 is the file descriptor. The 14 is all three `fwrite` calls, in one crossing.

[S3] `fflush(3)` — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>

fwrite takes a size and a count

Four arguments, and two of them multiply.

```
fwrite("three\n", 1, 6, f);  
/*      data      ^  ^  stream  
              |  +-- number of items  
              +----- size of one item, in bytes */
```

1 It returns a count

Items written, not bytes. The count equals bytes only when the item size is 1 [S21].

2 Fewer is legal

Below it, "a successful write() may transfer fewer than count bytes" [S2].

[S2] write(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/write.2.html> [S21] fread(3), which documents fwrite(3) — Linux man-pages. <https://man7.org/linux/man-pages/man3/fwrite.3.html>

Two operating systems, side by side

Read as a comparison. Try the tools yourself afterwards.

Process Monitor: what the kernel did

It reports filesystem and registry activity [S18], which is a different cut than the system-call boundary.

What it does

It shows "real-time file system, Registry and process/thread activity" [S18]. That view shows filesystem and registry activity, not system calls.

Where it comes from

Mark Russinovich wrote the page, in Microsoft's Sysinternals documentation [S18]. The tool "combines the features of two legacy Sysinternals utilities, Filemon and Regmon" [S18]. You download it. You can also run it from Sysinternals Live [S18].

HOW TO USE IT

```
Procmon.exe          # it starts capturing at once
Filter → Process Name is iodemo.exe # keep one process
Operation column     # expect CreateFile and WriteFile rows
```

[S18] Process Monitor. <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>

NtTrace: a tracer for the Windows Native API

It watches the native API, "the interface between the application space and the OS kernel" [S10].

What it does

A "simple trace facility for the Windows Native API", "roughly equivalent to strace on Linux" [S10]. It attaches as a debugger, so it "needs no driver and no administrator rights" [V21].

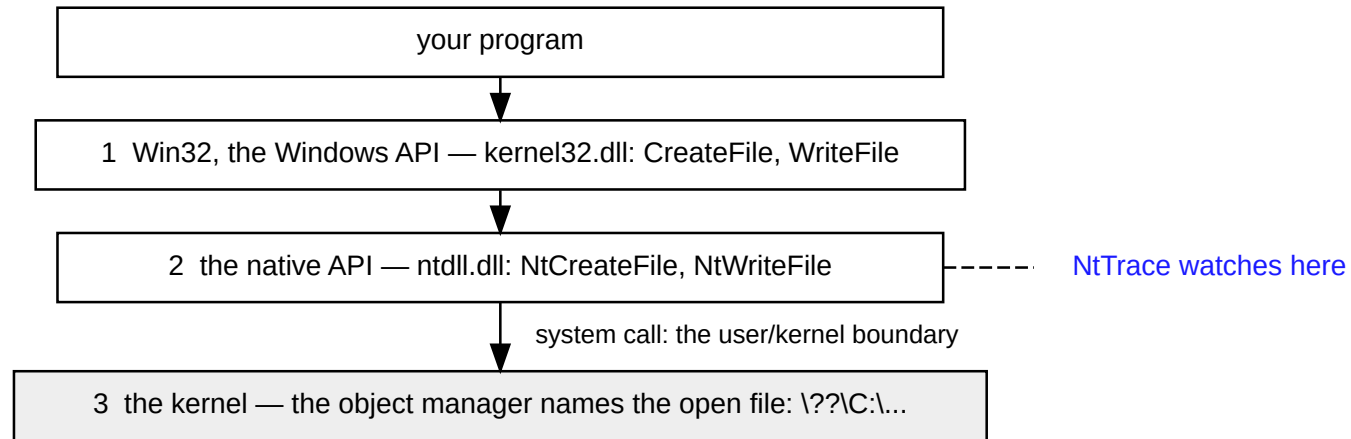
HOW TO USE IT

```
NtTrace iodemo.exe           # one line per native call
NtTrace -filter File iodemo.exe # keep calls whose names contain "File" [S10]
NtTrace -only iodemo.exe      # not its child processes [S10]
```

Try both yourself on `iodemo.exe`: Process Monitor shows what the filesystem did, NtTrace shows what the program asked for.

[S10] NtTrace — Roger Orr. <https://rogerorr.github.io/NtTrace/> [S33] NtTrace source. <https://github.com/rogerorr/NtTrace>

Three Windows words you will need



- 1 Win32** The documented interface a Windows program calls. The 32 is history, not a width [S22].
- 2 The native API** The layer below Win32, inside `ntdll.dll`. Microsoft calls it "internal to the operating system" [S9].
- 3 The object manager** The kernel component that names open objects. Its paths begin `\??\` [V22].

[S9] `winternl.h` — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation> [S22] Windows API index — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list>

Predict: the same program, on Windows

Before the Windows capture goes on the screen. Two answers, on paper [S14].

- 1 The three fwrite calls**
How many write calls does the Windows trace show — three, or one?
- 2 The deliberate failure**
Linux reported -1 ENOENT. How many error codes does Windows report for the same failed open?

The first answer tests whether buffering is a C library property or a platform property. The second one has a surprise in it.

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

A real Windows trace

You can already read this. The anatomy matches the Linux line, and the vocabulary differs.

```
NtCreateFile(... ObjectAttributes=0x58:"no-such-file.txt", ...)
    ⇒ 0xc0000034 [2 'The system cannot find the file specified.']
NtCreateFile(FileHandle=... [0x124], ... ObjectAttributes=0x58:"iodemo.out", ...) ⇒ 0
NtWriteFile(FileHandle=0x124, ..., Length=0xe, ...) ⇒ 0
NtClose(Handle=0x124) ⇒ 0
```

`iodemo.exe` under NtTrace, 2026-09-15 — the same four steps as the Linux trace [S36] [S37]. Ignore every argument except the handle, the file name and the return value.

[S36] Our NtTrace64 capture of `iodemo.exe`, 2026-09-15 — `demo/iodemo-c/NtTrace64-IODEMO-Output.txt` [S37] The container capture of `iodemo`, 2026-09-15 — `demo/iodemo-c/README.md`

A handle is not a file descriptor

First difference, visible in the first argument of almost every line.

LINUX

A file descriptor. A small integer indexing a per-process table. Our trace shows = 3 [S37].

WINDOWS

A handle. An opaque value naming an open object. Our trace shows
FileHandle=0x124 [S36].

Same job in a trace, different mechanism in the system. Do not use one word for the other.

[S36] Our NtTrace64 capture of iodemo.exe, 2026-09-15 — demo/iodemo-c/NtTrace64-IODEMO-Output.txt [S37] The container capture of iodemo, 2026-09-15 — demo/iodemo-c/README.md

Two error namespaces in one return value

Second difference, and it is the layer boundary showing up where nobody expects it.

```
⇒ 0xc0000034 [2 'The system cannot find the file specified.']
```

- | | | |
|----------|----------------------|---|
| 1 | First number | An NTSTATUS, the native layer's code [S36]. |
| 2 | Second number | The Win32 error it translates into [S36]. |
| 3 | Linux | One errno: -1 ENOENT [S37]. |

[S36] Our NtTrace64 capture of iodemo.exe, 2026-09-15 — demo/iodemo-c/NtTrace64-IODEMO-Output.txt [S37] The container capture of iodemo, 2026-09-15 — demo/iodemo-c/README.md

The stable boundary sits at a different level

Third difference, and the other two are symptoms of it.

In our trace of the Windows command shell a path reads "\\??\\C:\\WINDOWS\\system32\\c_852.nls" [S34]. That prefix "is the object-manager namespace, which a Win32 program never types" [V22].

LINUX

[V8]

The system
call interface
is the public
contract.

Syscall

WINDOWS

[S9]

Native entry
points are
"subject to
change from
one release of
Windows to
another".

Win32

[S9] winternl.h — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>

Where the boundary sits is contested history

Two readings survive the evidence. I will not pick one for you.

- 1** **Reading A: deliberate** POSIX, OS/2 and Win32 each reached NT through the native API [S19].
- 2** **Reading B: vestige** A reported POSIX subsystem that was "never really completed" [S15].
- 3** **Linux had no choice** It "had to make the kernel boundary itself the stable interface" [V38].

[S15] The sad history of the Microsoft POSIX subsystem — Reiter (2010). <https://brianreiter.org/2010/08/24/the-sad-history-of-the-microsoft-posix-subsystem/> [S19] Inside Native Applications — Russinovich (2006). <https://learn.microsoft.com/en-us/sysinternals/resources/inside-native-applications>

Linux is the outlier, not Windows

Only Linux promises that the raw system call interface stays the same [V9]. Say *different*, never *worse*.

- 1 Who makes no such promise** FreeBSD, OpenBSD, NetBSD, Solaris/illumos, macOS [V43].
- 2 Not promised is not broken** FreeBSD keeps old programs running through compatibility layers. It never writes the promise down [V36].
- 3 Why Windows and macOS can** They ship their own C library, so the library is the stable surface and the kernel boundary can move [V38].

One program, two kernels

One program, two systems, traced on one day.

STEP	LINUX [S37]	WINDOWS [S36]
failed open	<code>openat(... "no-such-file.txt" ...) = -1 ENOENT</code>	<code>NtCreateFile(... "no-such-file.txt" ...) ⇒ 0xc0000034 [2 '...']</code>
open	<code>openat(... "iodemo.out" ...) = 3</code>	<code>NtCreateFile(... [0x124] ... "iodemo.out" ...) ⇒ 0</code>
write	<code>write(3, "one\ntwo\nthree\n", 14) = 14</code>	<code>NtWriteFile(FileHandle=0x124, ... Length=0xe ...) ⇒ 0</code>
close	<code>close(3) = 0</code>	<code>NtClose(Handle=0x124) ⇒ 0</code>

Same program, same fourteen bytes, **one write on each side**. Hold the program fixed, and what is left is the two systems.

[S36] Our NtTrace64 capture of `iodemo.exe`, 2026-09-15 — `demo/iodemo-c/NtTrace64-IODEMO-Output.txt` [S37] The container capture of `iodemo`, 2026-09-15 — `demo/iodemo-c/README.md`

What we have not looked at

One question I will not answer, and the work that follows from it.

write() returned success. The bytes are not on a disk.

I built the whole hour towards this sentence. I leave it unresolved.

The manual says it plainly: "A successful return from write() does not make any guarantee that data has been committed to disk" [S2].

Saltzer, Reed and Clark gave the general form in 1984. A lower level may offer "an incomplete version of the function... useful as a performance enhancement" [S11].

So when is it actually safe? That is the durability lecture's question. I leave it open today.

[S2] write(2) — Linux man-pages. [S11] End-to-End Arguments in System Design — Saltzer, Reed and Clark (1984).
<https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf>

Buffering moves your errors

The error arrives somewhere other than where you caused it.

- | | | |
|---|---------------------------|---|
| 1 | Why it moves | The bytes only reached a buffer. |
| 2 | Where it arrives | Where the library flushes, so at <code>fclose</code> . |
| 3 | The rule | Check the return value of the call that flushes. |
| 4 | The idiom hides it | "the most common durability idiom in every mainstream language... contains no durability step anywhere" [V4]. |

Predict: all your writes succeeded — can the close still fail?

Yes or no, on paper. Then a second question: if it fails, is the descriptor still yours?

- | | | |
|----------|---|--|
| 1 | Every <code>write()</code> returned the byte count | The program did nothing wrong that it could see. |
| 2 | Now <code>close()</code> | Can it return -1? On what kind of filesystem? |
| 3 | If it did | Is the file descriptor number still open, closed, or unknowable? |

Two answers, written down [S14].

Why a close can fail when every write succeeded

The kernel deferred the write's error. The close surfaces it [S4].

- 1 Deferred** "errors on a previous write(2) operation are reported only on the final close()" [S4].
- 2 Where it bites** "silent loss of data ... with NFS and with disk quota" [S4].
- 3 The mirror** "A successful close does not guarantee that the data has been successfully saved to disk" [S4]. Part 6's question.

Against the stack: `write()` said *done* at level 3 [S2]. `close()` can still refuse [S4].

[S4] `close(2)` — Linux man-pages. <https://man7.org/linux/man-pages/man2/close.2.html>

Observe: close it twice, and watch what the second close hits

demo/close/ — one descriptor, two closes, both platforms [S39].

```
Linux, strace      close(3) = 0, then close(3) = -1 EBADF (Bad file descriptor)
Windows, untraced first CloseHandle ok, then error 6 (ERROR_INVALID_HANDLE)
```

The *first* close released the number, error or not: Linux "always releases the file descriptor early in the close operation, freeing it for reuse" [S4]. So a retry closes nothing — or, with threads, "a reused file descriptor from another thread" [S4].

Under a debugger `CloseHandle` "will throw an exception if it receives ... a handle value that is not valid ... This can happen if you close a handle twice" [S38]. *NtTrace* is a debugger, so the second close raises [S39].

[S4] `close(2)`. <https://man7.org/linux/man-pages/man2/close.2.html> · [S38] `CloseHandle`. <https://learn.microsoft.com/en-us/windows/win32/api/handleapi/nf-handleapi-closehandle> · [S39] demo/close/README.md. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/close/README.md>

Never retry a failed close

The obvious repair is wrong, and the manual page says so [S4].

"Retrying the close() after a failure return is the wrong thing to do, since this may cause a reused file descriptor from another thread to be closed" [S4].

A retry closes whatever now sits at that number, which may be another thread's file [S4].

Report the error. Then stop. Part 6 shows the same rule one level down, at fsync, and the data loss that taught it [V40].

[S4] close(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/close.2.html>

Networking and IPC are this same mechanism

Not new subjects. The same objects and the same calls, wearing different names.

1 **A socket on Linux** It is a file descriptor, and Linux makes that contract binding [V28].

2 **A socket on Windows** An opaque SOCKET, and Windows makes Winsock the contract [V28].

3 **A pipe** A file descriptor too, so the same calls reach it [V29].

Depth belongs to the next lecture. I want to stop you filing networking under "something else".

Honest limits

What today's tools cannot see.

- **The device level** [V5]
Not observable from software: no trace can check it.
- **Asynchronous I/O** [V11]
The tracer sees the setup, then no I/O.

Honest limits, continued

And what today's tools cost.

- **strace in production**

[V25]

Even the cheaper `--seccomp-bpf` mode [V39] "should therefore not be used on production systems!"

- **Mapping, crash dumps**

Each has its own lecture.

Lecture 2 ends here. The question does not.

`write()` returned success and the bytes are not on a disk [S2] — so when is it safe? Part 6 measures it.

Part 5 named the question and the honest limits of the tools. Your prediction for Part 6 has been on your page since Part 1: under a second, about five, about thirty, or not until shutdown.

PART 6

So when is it safe?

The question I left you with last week. Your prediction is already on your page.

The kernel's own count of what it has not written

No tracer here. The kernel publishes the number itself, and we watch it fall.

```
make container-watch          # demo/durability/  
# save.py writes 100 records, prints "done", exits  
# watch_dirty.py then samples /proc/meminfo every 0.5 s
```

Dirty

File data the kernel holds and has not written to the device yet. A page in this state is a **dirty page** [V78].

Writeback

Data the kernel is writing to the device right now [V78].

It is a machine-wide counter, so we watch it rise when `save.py` runs and fall when the kernel is finished [S52].

[S52] [demo/durability/README.md](https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md) — captured 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>

Observe: the program finished at zero seconds

```
vm.dirty_expire_centisecs    = 3000    (30 s)
vm.dirty_writeback_centisecs = 500     (5 s)

t= 0.0s Dirty=    1176 kB baseline
t= 0.0s Dirty=    5132 kB ← save.py has EXITED: "done: 100 records x 40960 B"
t= 0.5s Dirty=    5284 kB
t= 2.5s Dirty=    5364 kB
t= 4.0s Dirty=    5388 kB
t= 5.0s Dirty=     676 kB ← the kernel has finished
```

5.0 s

between the program's *done* and the kernel's, for a 4 MiB file [S52]

[S52] demo/durability/README.md — captured writeback timeline, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>

Five runs, three sizes

One run is an anecdote. Here are all five, including the one that disagrees.

WRITTEN	RUNS	THE KERNEL'S <i>DONE</i> , AFTER THE PROGRAM'S
$100 \times 40 \text{ KiB} = 4 \text{ MiB}$	3 (two on overlayfs, one on plain ext4)	5.0 s, 5.5 s, 5.5 s
$100 \times 64 \text{ B} = 6 \text{ KiB}$	1	5.5 s
$1000 \times 40 \text{ KiB} = 40 \text{ MiB}$	1	0.5 s

Four of the five runs land at five seconds, and file size barely matters between six kilobytes and four mebibytes [S52]. The forty-mebibyte run went out ten times sooner, which the next slide explains.

[S52] demo/durability/README.md — five runs, captured 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>

Five seconds in memory that belongs to nobody

The program's *done* and the kernel's *done* are different events, five seconds apart.

- 1** **Where the bytes were** In the **page cache**, level 3 of your drawing. That memory is the kernel's [V78].
- 2** **What set the delay** A kernel timer, not your program and not Python [V78].
- 3** **What a power cut takes** Everything in that window [V14].
The return value was correct. It reported that the kernel had accepted the bytes [V50].

Which rule fired, and how we know

The setting a student reads first is not the one that applied.

<code>dirty_expire_centisecs</code>	This machine: 3000, so 30 s [S52]. It marks when data is "eligible for writeout" [S68].
-------------------------------------	---

<code>dirty_writeback_centisecs</code>	This machine: 500, so 5 s [S52]. It sets how often the "kernel flusher threads ... wake up" [S68].
--	--

The 40 MiB run	Above <code>dirty_background_ratio</code> , "writeback starts sooner" [V48].
-----------------------	--

Writeback is the kernel copying dirty pages out. Which rule fired here is my reading.

[S68] [admin-guide/sysctl/vm.rst](https://www.kernel.org/doc/html/latest/admin-guide/sysctl/vm.html). <https://www.kernel.org/doc/html/latest/admin-guide/sysctl/vm.html> [S52] [demo/durability/README.md](https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md).
<https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>

Visibility is not durability

The objection you are about to make, and the answer to it.

VISIBILITY

After a successful `write()`, a later read returns the bytes. Another process reading the file sees them too. This is a coherence property of the page cache [V45].

DURABILITY

The bytes survive a power loss. This is a property of the medium [V45].

My own note calls confusing the two "the single most common error in this area" [V45]. So `cat results.csv` returning your data proves visibility. It proves nothing about the disk.

Predict: what does one `fsync` per record cost?

Before the counts. Same program, same hundred records, four ways [S14].

Measured

One `f.flush()` per record. 101 write calls, **1.1 ms** of wall time [S52].

Your prediction

Add one `os.fsync()` per record — the third arrow on the stack you drew, level 3 to level 4. How long?

Write a number in milliseconds. Not a factor, a number.

[S52] [demo/durability/README.md](https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md) — four-mode counts, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>

Observe: sixty times, for the same hundred writes

MODE	WRITE CALLS	FSYNC CALLS	WALL TIME
default	6	0	0.2 ms
flush per record	101	0	1.1 ms
fsync per record	101	100	67.3 ms
unbuffered	101	0	1.0 ms

Sixty times the wall time, for the same hundred write calls plus a hundred fsync calls [S52]. The cost is the waiting, not the extra crossings.

Microsoft concedes the same: FlushFileBuffers "can be inefficient when used after every write to a disk drive device" [S40]. Most programs never pay it.

[S52] demo/durability/README.md — four-mode counts, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md> [S40] FlushFileBuffers function (fileapi.h) — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-flushfilebuffers>

What `fsync()` promises, and what it does not

It crosses one boundary. The next is not yours to check.

- | | | |
|----------|---------------------|--|
| 1 | Does | Transfers the file's data "to the disk device", "including ... flushing a disk cache" [S67]. |
| 2 | Waits for | It "blocks until the device reports that the transfer has completed" [S67]. |
| 3 | Cannot check | Whether the device flushed. No trace tells "an honest flush from a fast lie" [V5]. |

Durability is a chain: each call flushes across one boundary, and each can fail alone [V44]. [The device reports.](#) [T1]

[S67] `fsync(2)` — Linux man-pages. <https://man7.org/linux/man-pages/man2/fsync.2.html>

Windows: the same shape, and now measured

The Windows twin of the four-mode run, on my Windows 11 VM [S52].

IOCost.exe, 100 × 4 KiB	KERNEL WRITE OPS	WALL
WriteFile into the cache	100	0.7 ms
FlushFileBuffers per record	100	27.9 ms
FILE_FLAG_WRITE_THROUGH	100	25.2 ms
FILE_FLAG_NO_BUFFERING	100	9.9 ms

[Forty times](#), for asking the device to confirm each record [S52]. Linux's figure was sixty. The two ratios are not comparable: these records are 4 KiB, the Linux ones were 64 B [S52].

Now read the count column: 100 in every row.

[S52] demo/durability/README.md, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md> [S40] FlushFileBuffers — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-flushfilebuffers>

The two kernels write in opposite orders

The same watch, on Windows, until the counter returned to baseline [S52].

WRITTEN	WINDOWS CACHE MANAGER	LINUX KERNEL
4 MiB, three runs	0.5 s · 1.0 s · 0.5 s	5.0 s · 5.5 s · 5.5 s
40 MiB, one run	14.1 s	0.5 s

Windows' **lazy writer** had the small file out inside a second. Linux waited for its flusher's five-second wake. On the large file they swap — Linux wrote it at once, Windows trickled 512 pages a pass [S52].

Both clocks are a kernel's. Neither is your program's.

[S52] demo/durability/README.md, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>

Predict: your `fsync()` failed — what does the retry return?

The device rejected the write, so `fsync()` returned `-1` with `EIO`. The obvious repair: call it again.

1 **The second call** `-1` again · 0, bytes on the device · 0, bytes gone.

2 **Who found out** A database team, in 2018, in production [S41].

One of three, written down next to your Part 1 prediction [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/> · [S41] `fsyncgate`. <https://danluu.com/fsyncgate/>

Observe: the retry returns success, over data that was never written

2018, and the case is known as *fsync-gate*. A PostgreSQL developer found that retrying a failed `fsync()` "is not OK at least on Linux" [S41].

```
fsync(3) = -1 EIO      ← the kernel reports the failure, once
close(3)
open(...)
fsync(3) = 0          ← success, over data that was never written
```

That is a real reproduction against an error-injecting device. Its author annotated the last line "fsync on file with still-dirty data+EIO returns success BAD", for a byte that "certainly never got written" [V53].

[S41] fsyncgate — the pgsql-hackers thread, archived. <https://danluu.com/fsyncgate/>

Explain: the retry does not retry

Mechanical, not sloppy code: **the retry does not re-attempt the failed I/O** [V54].

1. **Reported once.** The kernel marks the page failed, reports it to the first `fsync()`, and clears the mark [S41].
2. **Already gone.** A failed writeback drops the data and marks the page clean, so a retry has nothing to write [V54].
3. **So the retry lies.** "It reads a status register that the previous read destroyed, over a page cache the data has already left" [V89].

The result: "how PostgreSQL lost data silently for twenty years" [V90].

What the engines did instead

Three storage engines, three architectures, one answer, in one year.

POSTGRESQL	[V40]	Retry replaced by a crash [V93].	2018
INNODB	[V84]	Retry loop became a fatal error.	2018
WIREDTIGER	[V24]	No retry. Panic on a failed flush.	2018

A stopgap that "has now lasted eight years" [V56].

Windows here is not "the same", and not "safe"

It is undocumented, which is a third answer and a worse one.

- **What is missing**

[V47]

The PostgreSQL wiki's matrix of post-fsync-failure behaviour has no Windows row at all.

- **Why nobody can settle it**

[V55]

The Linux contract was inspectable, so it was discoverable. The Windows contract "cannot be settled by reading" [V55].

Linux documents a hostile contract: one error report, then the data is gone. That is actionable. "For a system that must not lose data, an unspecified contract is the worse of the two positions" [V91].

Sync the directory too

This part is yours, and the classic recipe is incomplete [V51].

A file is two pieces of data in two places: its **bytes**, and its **name**, which lives in the directory. `fsync()` on the file flushes the bytes. The manual: "Calling `fsync()` does not necessarily ensure that the entry in the directory containing the file has also reached disk" [S67].

So "create-write-fsync-close on a new file is the classic incomplete recipe" [V51]: after a crash the bytes are safe and the file may have no name.

The fix costs one line: open the directory, and `fsync()` that descriptor too [S67].

Write-then-rename, and why it needs a sync

How every editor saves, and how ext4 made it lose files in 2009 [S44].

```
write a temporary file → close it → rename(temp, original)
```

Why the pattern. `rename()` swaps the name atomically: "there is no point at which another process attempting to access newpath will find it missing" [S70].

The trap. The rename is a small piece of metadata. The temporary file's bytes are still dirty in the page cache. ext4 wrote the rename first, so a crash meant "that existing file gets replaced with a zero-length file" [S44].

The fix. `fsync()` the temporary file *before* the rename [S44]. Then the name can only point at bytes already on the device.

The respectable opposite view

Most programs are right not to pay, until the failure is "*silent*" [V52].

- 1 Most need not pay**

SQLite's own write-ahead-log setting "does lose durability", yet "a good choice for most applications" [S43].
- 2 Trusting the layer**

Trusting the layer below is "a defensible ... position", and "the implicit default" [V52].
- 3 Blame the API**

"no one ever, ever wrote `creat()`; `write()`; `close()`; `rename()`; and hoped they would get an empty file" [S45].

[S43] SQLite. https://www.sqlite.org/prAGMA.html#prAGMA_synchronous [S45] Aurora, LWN (2009). <https://lwn.net/Articles/351422/>

Log first — and what if the log write fails?

The write-ahead log protects you because of an *order*, and the order covers its own failure.

1. **Order.** Log first, `fsync`d. Only then *committed* [V85]. The data file follows later, at a checkpoint [S71].
2. **Log `fsync` fails:** nothing was promised yet. PostgreSQL panics here too: "we already did the same for WAL data" [S71].
3. **Data-file `fsync` fails later:** "the only remaining copy of the data is in the WAL" [S71]. Crash, replay.

The log wins only because it is made durable *first*, one `fsync` per commit — the 67 ms you saw [S52].

So when is it safe?

The answer, and what it does not cover.

In software

The last *done* you see is `fsync()`, resting on the device's report [S67].

In hardware

A tested UPS keeps the host up. Level 4 is inside the drive, and needs its own power-loss protection.

Neither covers

`fsync-gate` lost data **with the power on** [V53]. No battery helps.

So also

Log every change first: "recover from the WAL after any failure" [V85].

SQLite trusts neither call — "neither ... works as advertised" [V49].

Lecture 4 starts here

Parts 7, 8 and 9 are built in full. If the clock has gone, Lecture 4 opens on this slide and we jump to part 10.

What a pipe is

The thing we are about to wait on, a hundred times over.

"Pipes and FIFOs (also known as named pipes) provide a unidirectional interprocess communication channel. A pipe has a *read end* and a *write end*" [S59].

- A pipe is a 64 KiB buffer inside the kernel [S59]. One process writes into one end, another reads from the other.
- Both ends are file descriptors, so `read()` and `write()` work on them unchanged [S59].
- "If a process attempts to read from an empty pipe, then `read(2)` will block until data is available" [S59]. That word, *block*, is the next slide.

Blocking, non-blocking, and letting the kernel wait for you

Three ways to ask a pipe "is there anything to read?"

1. **Blocking.** `read()` on an empty pipe parks your thread until data arrives [S59]. Fine for one pipe, hopeless for a hundred.
2. **Non-blocking.** Set `O_NONBLOCK`. Now `read()` returns at once, with `EAGAIN`, when there is nothing [S59]. Ask all hundred, again and again: *polling*.
3. **Let the kernel wait.** `select()` and `poll()` take a list of descriptors and sleep until one is "ready", meaning a read "without blocking" is possible [S47]. One sleeping call instead of thousands of asks. `epoll` is Linux's faster form of the same idea [S46].

Predict: a hundred pipes, and nobody writing

Two programs, two seconds, zero bytes of data. They differ only in how they wait [S14].

The setup

A hundred pipes. Nobody ever writes to them. Both programs wait two seconds, then exit.

Program A

Asks every pipe in turn, non-blocking, round after round.

Program B

Registers all hundred once, then makes one call that sleeps.

The question

How many system calls does each one make? *About a hundred · a thousand · thirty thousand · millions.*

Two numbers, on paper. Neither program transfers a single byte.

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Where the question comes from

The cost of waiting, not of I/O.

KEGEL, 1999	[S55]	"So hardware is no longer the bottleneck."	Named
GAMMO AND COLLEAGUES	[S49]	Plus 10,000 idle connections: "degrade by up to 79%".	Measured

Up to 79% of select/poll throughput gone, only idle connections added [V61]. No answer made a read faster: each changed "**where the interest set lives**" [V62].

Kegel's 1999 revision lacks the idleness clause everyone quotes [S55] [V63].

[S55] The C10K problem, 8 May 1999 — Kegel. <https://web.archive.org/web/19990508164301/http://www.kegel.com/c10k.html> [S49] Gammo et al. <https://www.kernel.org/doc/ols/2004/ols2004v1-pages-215-226.pdf>

A thread each, and why that stopped working

A hundred pipes, a hundred blocking threads — *thread-per-connection* — failed on two bills [S59].

1 Memory

Kegel priced the stacks: 2 MB each, "not an uncommon default value", against 1 GB of user-accessible address space. 512 threads, then no virtual memory left [V94].

2 Scheduler

Banga, Mogul and Druschel priced the rest: it "scales badly as the number of threads increases" [V95].

Either way the bill is per connection, not per byte [V96]. My note calls that "the same mistake at two altitudes" — my reading, not either source's [V96].

Observe: two seconds, zero bytes, counted

wait_blocking.py				wait_epoll.py			
% time	calls	errors	syscall	% time	calls	errors	syscall
97.75	28017	28000	read	16.97	100		epoll_ctl
				1.97	1		epoll_pwait

28,017

read calls, 28,000 of them failing
with EAGAIN [S53]

1

epoll_pwait call, which slept for the
whole two seconds [S53]

EAGAIN is not a fault. It is the kernel answering "nothing yet" to a non-blocking read, and `strace -c files` it under errors [S53].

[S53] demo/waiting/README.md — captured 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/waiting/README.md>

What those counts do and do not say

Three things about 28,017 and 1 before you carry them anywhere [S53].

1. **28,017 grows with time.** Twenty seconds of polling makes ten times the calls, and every one of them learns nothing [S53].
2. **1 does not.** The `epoll` program paid a hundred registrations once, then one wait, however long it waits [S53].
3. **28,017 is too low.** The tracer slows every call. Untraced, the same loop "asked 1.3 million times in one second" [S53].

So: count under the tracer. Time without it [S1]. Brendan Gregg measured a loop of nothing but system calls running 442× slower under `strace` [S66] [V17].

epoll is three calls

1. `epoll_create1` makes an **interest set** that lives in the kernel between calls [S46].
2. `epoll_ctl` adds or removes one descriptor. Our hundred registrations [S53].
3. `epoll_wait` sleeps until a registered descriptor is readable. Our one call [S53].

`select` and `poll` hand the kernel the whole list on every call [S46]. `epoll` states it once, which is why Program B's count was a constant.

Readable has a precise meaning: a read "without blocking" is possible, because at least one byte is waiting or the writer has closed its end [S47]. It does not say how many bytes.

Level-triggered and edge-triggered

One flag. Get it wrong and a connection goes quiet.

LEVEL-TRIGGERED — THE DEFAULT

Unread data is reported on every wait. The manual: "a faster `poll(2)`" [S46].

EDGE-TRIGGERED — EPOLLET

Reports only when something *new* arrives [S46]. Read 1 kB of a 2 kB arrival and the next wait "will probably hang" [S46].

The rule for EPOLLET: every time you are woken, read everything — until EAGAIN — because the kernel will not mention the rest again [S46]. Non-blocking descriptors, or that loop never ends [S46].

Three ways to wait, and we watch all three

One ladder. Each rung answers "who gets parked, and where" [V69].

- | | | |
|---|-------------------|---|
| 1 | Polling | The program asks each descriptor. Our 28,000 failed reads [S53]. |
| 2 | Readiness | The kernel says "you may read now" [S46]. Our one <code>epoll_pwait</code> [S53]. |
| 3 | Completion | The program hands over a buffer. The kernel says "I have read for you" [V74]. |

In a trace: many failed reads means **polling**, one sleeping call means **readiness**, no reads at all during the wait means **completion**.

[S46] `epoll(7)` — Linux man-pages. <https://man7.org/linux/man-pages/man7/epoll.7.html>

Readiness is permission to try, not a promise

The word *ready* sounds like a guarantee. It is not one.

`select(2)`, BUGS: it "may report a socket file descriptor as "ready for reading", while nevertheless a subsequent read blocks" [S47].

1 So use non-blocking mode

It is "safer to use `O_NONBLOCK` on sockets when blocking is undesirable" [S47].

2 The model

"permission to try, not a promise of success" [V58].

Ready reports a moment that has passed, on `select` and on `epoll` alike [V92]. A *done*, one rung up.

[S47] `select(2)` — Linux man-pages. <https://man7.org/linux/man-pages/man2/select.2.html>

The real difference is who owns the buffer

Not the API shape. The ownership, and it is why porting between the models is a rewrite.

	READINESS	COMPLETION
when you supply the buffer	after you learn the descriptor is ready [V75]	at submission, before any data exists [V74]
who owns it while waiting	you — the kernel holds no pointer [V75]	the kernel — you must not touch or free it [V74]
one buffer can	be shared across ten thousand connections [V75]	serve one outstanding operation [V74]

My note calls ownership "the load-bearing row" [V57]. Microsoft's contract: clients "agree not to disturb the buffers in any way until the network has consumed the buffer's contents" [V74]. [T2]

It is not a Linux/Windows split

I have used the platforms as labels. Here is the correction.

- 1 The honest statement** "the readiness/completion split is not a Linux/Windows split" [V59].
- 2 The control case** BSD and macOS use kqueue, which is readiness [V81]. Not a Linux quirk [V59].
- 3 And both have crossed** Linux built a completion interface, Windows built a shared-memory ring [V68].

The useful question is not which system a program runs on. It is [which *done* it is handed](#).

The trap this demonstration sets

We measured pipes. A regular file behaves differently, and the difference is silent.

- **Readiness and regular files**

[V73]

`epoll` "has nothing useful to say about a regular file, which is always "ready"". The call returns at once and you learn nothing.

- **Why a completion model exists at all**

[V88]

Linux "grew a completion interface because readiness could not express what its users needed".

So when you do meet these interfaces, do not expect a disk file in an `epoll` set to wait. It will not.

Four Windows words before the Windows pair

The same two programs, in Windows vocabulary [S53].

1. **Named pipe.** "a named, one-way or duplex pipe for communication between the pipe server and one or more pipe clients" [S72]. A pipe with a name, openable by unrelated processes.
2. **PeekNamedPipe.** Looks without reading: it copies data "without removing it from the pipe" and "always returns immediately" [S73].
3. **Overlapped ReadFile.** A read that returns before the data exists. The kernel fills your buffer later [V74].
4. **Completion port, GetQueuedCompletionStatus.** A queue of finished operations. The call takes one, and "If there is no completion packet queued, the function waits" [S74].

Predict: the same hundred pipes, on Windows

Completion is the rung we have not yet watched run [S14].

Program C

A hundred named pipes. PeekNamedPipe on each, round after round — the Windows polling twin.

Program D

One overlapped ReadFile posted per pipe, all bound to one completion port, then one GetQueuedCompletionStatus that sleeps.

The question

Under NtTrace, how many native calls does each make in two seconds — and how many *reads* does Program D do while it waits?

Two numbers again. Both programs are in `demo/waiting/` [S53].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/> [S53] `demo/waiting/README.md`, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/waiting/README.md>

Observe: Windows, traced, both ways

wait_peek.exe	8153 native calls	wait_iocp.exe	956 native calls
2600	NtFsControlFile	100	NtReadFile (submitted up front)
2500	NtCreateEvent	100	NtSetInformationFile
2725	NtClose	1	NtCreateIoCompletion
		1	NtRemoveIoCompletion ⇒ STATUS_TIMEOUT

3

native calls per PeekNamedPipe:
create an event, control the pipe,
close [S53]

1

NtRemoveIoCompletion, which slept
the whole two seconds [S53]

Untraced, the polling program made **330,100** peeks in two seconds. Traced, it made 2,500 [S53]. Count under the tracer. Time without it.

[S53] demo/waiting/README.md — NtTrace64 captures, Windows 11 build 26200, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/waiting/README.md>

Which numbers you may compare, and which you may not

Two counts per platform. Compare within a platform, never across [S53].

You may, and shall, compare within one platform, both runs traced:

- Linux: 28,017 read calls against 1 `epoll_pwait` [S53].
- Windows: 8,153 native calls, three per peek, against 956, of which one `NtRemoveIoCompletion` slept the whole two seconds [S53].

You may not compare 28,017 with 330,100, since one is traced and one is not [S53]. Nor Linux with Windows: different machines, and read is not `PeekNamedPipe` [S53].

What survives: the same shape, twice. Thousands of asks against one sleeping call, on two kernels that agree on little else.

What a ring is

One word before the last slide of Part 7.

A ring is two queues in memory that your program and the kernel both see: "ring buffers which are shared between user space and kernel space" [S75]. "You place I/O requests you want to make on the SQ, while the kernel places the results of those operations on the CQ" [S75].

Why it matters here: every wait we counted today was a system call. With a ring you queue many requests and cross the boundary once [S75], or, with a kernel-side polling thread, "steady-state I/O submission needs **no syscalls at all**" [V11].

Where both platforms ended up

Not convergence. Each one took the half it was missing.

LINUX

Readiness-native. Built a completion interface, `io_uring`, in kernel 5.1 [V79].

WINDOWS

Completion-native since NT [V80]. Built a ring, `IoRing`, in Windows 11 build 22000 [V80].

"Linux adopted completion semantics; Windows adopted the shared-memory transport" [V68]. The ring and the completion model are two separable ideas, and each platform took the one it lacked.

Learn IOCP, the I/O completion port, as current Windows practice. Rings are "the direction of travel", not the exam [V60].

Same mechanism, different clothes

A file, a pipe and a socket. One program, one trace, three destinations.

Predict: which of the four calls changes?

You know the file run by heart now. I change only the destination [S14].

```
openat(AT_FDCWD, "no-such-file.txt", O_RDONLY) = -1 ENOENT
openat(AT_FDCWD, "iodemo.out", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
write(3, "one\ntwo\nthree\n", 14)      = 14
close(3)                                = 0
```

I now run `./iodemo /dev/stdout | cat`. The bytes go into a pipe and cat reads them.

On paper

Which of the four calls changes — the `openat`, the `write`, the `close`, or nothing?

And

Does the byte count in the `write` change?

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Observe: the file run and the pipe run

```
= to a file =
openat(... "no-such-file.txt" ...) = -1
openat(... "iodemo.out" ...)      = 3
write(3, "one\ntwo\nthree\n", 14) = 14
close(3)                          = 0

= to a pipe =
openat(... "no-such-file.txt" ...) = -1
openat(... "/dev/stdout" ...)      = 3
write(3, "one\ntwo\nthree\n", 14) = 14
close(3)                          = 0
```

The same four calls, the same fourteen bytes in one write, the same descriptor number [S54]. cat on the other end received one two three.

The `openat` argument changed. The call did not, the count did not, and the byte total did not [S54].

[S54] demo/iodemo-c/README.md — the pipe run, captured 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/iodemo-c/README.md>

The descriptor is the abstraction, not the file

The reason is a kernel structure, not a convention.

"On Linux, anything that takes a file descriptor takes a socket, because the kernel routes both through the same dispatch table" [V64]. A pipe too [V29]. [T3]

So `write(3, ...)` said *done* when the bytes were in the pipe. Had `cat` read them yet? Probably not — but the capture covers only the writer, so that is my reading.

1

write call, for a file and a pipe alike
[S54]

[S54] `demo/iodemo-c/README.md`, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/iodemo-c/README.md>

Predict: the same program, writing to your terminal

`./iodemo /dev/stdout` with nothing after it. No pipe, no file. The screen [S14].

The same three `fwrite` calls: `"one\n"`, `"two\n"`, `"three\n"`. The file run and the pipe run each made **one** `write()` of fourteen bytes [S54].

On paper: how many `write()` calls when the destination is a terminal — one, or three? And a second answer: who decides, the kernel or the C library?

Observe: three writes, not one

```
openat(AT_FDCWD, "/dev/stdout", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
write(3, "one\n", 4)           = 4
write(3, "two\n", 4)          = 4
write(3, "three\n", 6)        = 6
close(3)                     = 0
```

Same binary, same container, run with a terminal attached (podman run -t). Captured 2026-09-21 [S54]. Three system calls where the pipe run made one. The program did not change. The C library's decision did.

Explain: the destination changed the trace, at the C library, not the kernel

"Attached to a terminal, stdout is line-buffered... Attached to a pipe or a regular file, stdout is fully (block) buffered" [V71].

1. **The library asks once**, at the first write, what the descriptor is. A terminal gets line buffering: every `\n` becomes a `write()` [V71].
2. **A file and a pipe** sit on the same side of that rule, so the first two runs agreed *by construction*. Both destinations are ones the library treats the same.
3. **The kernel's part never changed**: `openat`, `write`, `close`. Only how many times.

Try it yourself: `./iodemo /dev/stdout` in a terminal, then with `| cat` [S54].

Predict: the network leg

Same fourteen bytes, over a TCP socket, with netcat on both ends [S14].

The setup

`./iodemo /dev/stdout | nc 127.0.0.1 9000`, with `nc -l` listening.
Both netcats traced.

The question

The listener receives the fourteen bytes from a TCP socket. Which call takes them off the wire?

Two options

One of two. Write it down.

`read`, the same call you would use for a file — or a socket-specific call.

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Observe: both netcats, traced

```

= client nc (writer) =
socket(AF_INET, SOCK_STREAM...) = 3
connect(3, ...9000...) = -1 EINPROGRESS
read(0, "one\ntwo\nthree\n", 16384) = 14
write(3, "one\ntwo\nthree\n", 14) = 14
close(3) = 0

= listener nc (reader) =
socket(AF_INET, SOCK_STREAM...) = 3
bind(3, ...9000...) = 0
listen(3, 1) = 0
accept4(3, ...) = 4
read(4, "one\ntwo\nthree\n", ...) = 14

```

The client puts them on the wire with `write(3, ...)`. The listener takes them off with `read(4, ...)`. The same two calls `iodemo` used for a file and for a pipe [S54].

What is new is only how the descriptor came to exist [S54].

[S54] [demo/iodemo-c/README.md](https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/iodemo-c/README.md) — the traced netcat pair, captured 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/iodemo-c/README.md>

Then what are send and recv for?

They are the program's choice of spelling, not the kernel's.

send(2) settles it in its own opening: "The only difference between send() and write() is the presence of flags. With a zero flags argument, send() is equivalent to write(2)" [S50].

- 1 Use them for a flag** MSG_DONTWAIT and MSG_NOSIGNAL have no write() equivalent.
- 2 Otherwise it is style** They tell a human reader "this is a socket".

[S50] send(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/send.2.html>

The same interface is not the same contract

Three boundaries on the result.

- **One send is not one receive** [V70]
TCP is a byte stream. Marking where a message ends is your job — "the single most common practical bug in student socket code".
- **A size effect** [V82]
"writes of less than PIPE_BUF bytes must be atomic" — 4 KiB here [V67] [V82].
- **The default is not total** [V87]
"a strong default, not a total function" [V65]. `lseek` on a socket means nothing.

Predict: the same three destinations, on Windows

On Linux the destination changed nothing in the trace. Windows is the contrast [S57].

- 1** **Pipe** WriteFile to an anonymous pipe. Fourteen bytes.
- 2** **Socket** send() on a loopback TCP socket, fourteen bytes.
- 3** **The same socket again** WriteFile on that socket, cast to a HANDLE.

On paper: **is Windows the same as Linux here: only the destination changes, the call stays the same?**
Yes or no, and if no, which leg differs? One program, three destinations [S57].

[S57] demo/destinations-win32/README.md — captured 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/destinations-win32/README.md>

Observe: the API changes the call, the object does not

```
NtWriteFile(FileHandle=0x118, Length=0xe, IoStatusBlock [0/0xe]) ⇒ 0
NtDeviceIoControlFile(FileHandle=0x138, IoControlCode=0x0001201f, ...) ⇒ 0
NtWriteFile(FileHandle=0x138, Length=0xe) ⇒ 0
```

1 · PIPE	[S57]	NtWriteFile, 14 bytes	file path
2 · SEND()	[S57]	NtDeviceIoControlFile, control code 0x1201f	device control
3 · WRITEFILE	[S57]	NtWriteFile — same handle as leg 2	file path

Legs 2 and 3 are one handle, two native calls. A plain WriteFile first failed with error 87 — Winsock sockets are **overlapped handles** [S57].

[S57] demo/destinations-win32/README.md, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/destinations-win32/README.md>

Windows: a SOCKET is not a HANDLE

Third time today, and the same asymmetry each time.

LINUX

A socket is a file descriptor, and Linux makes the file-descriptor contract binding [V28].

WINDOWS

An opaque SOCKET, and Windows makes Winsock the contract [V28].

The NT kernel does hold a file object underneath [V28] — which is exactly what leg 3 just wrote to [S57]. The *type* is not a C-runtime file descriptor, while the *handle* is a kernel file handle [V66].

So the platforms agree on the mechanism and disagree on which layer they promise. Third time today.

Bytes you never touch

One call can carry several buffers. And the kernel can move a file your process never holds.

Predict: three buffers, one message

A message that is three pieces in your program, and one message in the file [S14].

The program

A header, a body and a trailer — three separate buffers. You want all three in the file, in order, as one message.

Question one

How many system calls does that take — *three, one, or zero?*

Question two

How many times does anything copy the three buffers to make it happen?

Two answers, written separately. They are not the same question [S56].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/> [S56] demo/scatter/README.md — captured 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md>

Observe: three lines become one

```
= three write() calls
write(3, "HEADER  ", 8)                = 8
write(3, "body body body body body body bo"... , 40) = 40
write(3, " TRAILER\n", 9)              = 9

= one writev() call (wrapped to fit)
writev(3, [{iov_base="HEADER  ", iov_len=8},
           {iov_base="body body body body body body bo"... , iov_len=40},
           {iov_base=" TRAILER\n", iov_len=9}], 3) = 57
```

Three crossings became **one**, and the return value is the whole message: 57 bytes [S56]. The three buffers never moved in the program.

[S56] demo/scatter/README.md — strace -e trace=write,writev, 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md>

Gather buys crossings, not copies

The distinction the name hides, and your second answer.

WHAT WRITEV REMOVES

Two of the three user-to-kernel crossings, plus the staging buffer you would glue the pieces into [S58].

WHAT IT DOES NOT REMOVE

Any copy the kernel makes further down. The four levels underneath are unchanged [S56].

`readv` is the mirror: one crossing *scatters* into several buffers [S58]. Valkey's reply path gathers framing, a stored string and a terminator into three `iovec` entries, then sends them with one `writv` [S65]. [T4]

[S58] `writv(2)` — Linux man-pages. <https://man7.org/linux/man-pages/man2/writv.2.html> [S56] [demo/scatter/README.md](https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md), 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md> [S65] [valkey/src/networking.c](https://github.com/valkey-io/valkey/blob/00a8a19554a39facb591837ea41ba4350260919d/src/networking.c) — Valkey (2026). <https://github.com/valkey-io/valkey/blob/00a8a19554a39facb591837ea41ba4350260919d/src/networking.c>

Two promises, and only one of them is unconditional

Atomic means one thing: your bytes land as one unbroken block, with nobody else's write in the middle [S58].

1. **On a regular file** the promise is unconditional: "written as a single block that is not intermingled with output from writes in other processes" [S58]. A log line written with one `writew` stays whole.
2. **On a pipe** it holds only below `PIPE_BUF`, 4096 bytes. Above that the kernel "may interleave the data" with another writer's [S59].
3. **The trap:** the limit applies to the *sum* of your buffers, not the largest. Eight 1 KiB pieces are 8 KiB, and may interleave.

Predict: moving a file you never look at

4 MiB from a file into a pipe. Two programs, same bytes [S14].

Program A

A copy loop: read 64 KiB into a buffer, write it out, repeat.

Program B

One call per chunk: `sendfile(out, in, ...)`. The program never allocates a buffer for the data.

The question

How many data system calls does each make — and how many times does each byte cross into your address space?

Neither program looks at a single byte of the file [S56].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/> [S56] `demo/scatter/README.md` — captured 2026-09-20. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md>

Observe: half the calls, and no trip through your program

4 MIB INTO A PIPE	DATA SYSCALLS	COPIES THROUGH USER SPACE	WALL
copy loop, 64 KiB chunks	64 read + 64 write = 128	two per chunk	2.6 ms
sendfile	64 sendfile	none	2.0 ms

The trace's tell is an absence: with `sendfile` there is **no read line at all** [S56]. The bytes went from the page cache to the pipe without visiting your address space.

Sixty-four calls because each moved 64 KiB — the pipe's capacity, not a `sendfile` limit [S56] [S59].

[S56] [demo/scatter/README.md](https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md) — `strace -c -e trace=read,write,sendfile, 2026-09-20`. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md> [S59] `pipe(7)` — Linux man-pages. <https://man7.org/linux/man-pages/man7/pipe.7.html>

What the name does not cover

It counts the copies it removes at one boundary. No others.

- | | | |
|----------|-----------------------------------|---|
| 1 | Direct memory access stays | The bytes still move. The CPU no longer carries them [S60]. |
| 2 | It needs the page cache | Level 3 is not bypassed. Handing out its pages <i>is</i> the mechanism. |
| 3 | It buys no durability | The same four levels sit below [S67]. |
| 4 | It cannot help you look | Encrypt them and the process must hold them [S60]. |

[S60] sendfile(2). <https://man7.org/linux/man-pages/man2/sendfile.2.html> [S67] fsync(2). <https://man7.org/linux/man-pages/man2/fsync.2.html>

Zero-copy moves bytes faster. It does not make them safer.

Fewer copies changes *who* copies, not *when the bytes are safe*.

1. **To a file.** `sendfile` copies "within the kernel" [S60], into the page cache. Nothing is on the device until `fsync()` [S67].
2. **To a socket.** The return counts bytes the kernel accepted, not delivered: "No indication of failure to deliver is implicit in a `send()`" [S50].
3. **One new duty.** The kernel may still be reading *your* pages after the call returns: keep the file "unmodified until the reader on the other end of `out_fd` has consumed the transferred data" [S60].

Done still means *accepted* [V44].

Windows, and a silent fallback

Other names, and a Python fallback that says nothing.

- | | | |
|----------|-------------------------|---|
| 1 | Gather, a socket | WSASend with an array of WSABUF [S69]. |
| 2 | Gather, a file | WriteFileGather, but every buffer must be at least an aligned page [S62]. |
| 3 | Zero-copy | TransmitFile [S63]. |

Without `os.sendfile()`, `socket.sendfile()` uses "send() ... as a less efficient fallback" [S64]. **No warning:** on Windows you measure that.

[S62] WriteFileGather — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-writefilegather> [S63]
TransmitFile — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/mswsock/nf-mswsock-transmitfile> [S64] socket — Python docs. <https://docs.python.org/3/library/socket.html#socket.socket.sendfile>

Homework, and the next question

What changed about the assignment, what you can run this week, and where Lecture 4 picks up.

Assignment 1 is not set today

I told you 22 September in Lecture 1. That has moved, and here is the honest reason.

What changed

The submission infrastructure is not ready. The assignments move back.

New dates

None yet. I will not put a date on a slide before I can keep it.

What it will measure

Buffered against unbuffered, and sequential against random access, over a 1 MiB file. Predict the ratio. Measure it. Explain it in crossings.

Same method

Predict, observe, explain [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

Voluntary homework, and three rules for it

Everything I ran today ships with this deck. Run it yourself.

- 1 Predict first** Write the number first. Watching alone teaches nothing [S51].
- 2 Report version and buffer size** Python's default binary buffer moved from 8 KiB to 128 KiB [S7].
- 3 Count under the tracer** Then time it again, without the tracer [S1].
`demo/durability/ · demo/waiting/ · demo/iodeemo-c/ · demo/scatter/ · demo/close/` run in the container. `demo/destinations-win32/` needs Windows [S57].

[S51] Crouch, Fagen, Callan and Mazur (2004). <https://works.swarthmore.edu/fac-physics/203/> [S7] CPython gh-117151. <https://github.com/python/cpython/issues/117151> [S1] strace(1). <https://man7.org/linux/man-pages/man1/strace.1.html>

Summary — what follows from this

1. **Acknowledgement is not durability** [T1] — a return value names the layer that took the bytes [V50].
2. **Readiness or completion** [T2] — one is permission to try [V58], one holds your buffer [V74].
3. **One mechanism, three destinations** [T3] — one write carried a file, a pipe and a socket [S54].
4. **Crossings are not copies** [T4] — `writen` removed crossings, `sendfile` removed copies [S56].

Name the boundary first: "I flushed" is ambiguous across four [V44]. Every number came from a run [S52] [S53] [S56]. Whose *done* did you measure? [V50]

What Lecture 4 answers

Same as last week. I name it and do not answer it.

Today's window

Your bytes sat in the page cache for five seconds, in memory no process owns [S52].

In part 9

The kernel moved four mebibytes you never held — the same pages [S56].

The question

What is that memory, and whose is it?

Why it is hard

The kernel reclaims those pages on demand, so free memory is a cache [V72].

Lecture 4 opens with parts 7 to 9 if we miss them.

[S52] demo/durability/README.md. <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>

References · 1/27

- **[S1]** `strace(1)` — Linux man-pages — <https://man7.org/linux/man-pages/man1/strace.1.html>
- **[S2]** `write(2)` — Linux man-pages — <https://man7.org/linux/man-pages/man2/write.2.html>
- **[S3]** `fflush(3)` — Linux man-pages — <https://man7.org/linux/man-pages/man3/fflush.3.html>
- **[S4]** `close(2)` — Linux man-pages — <https://man7.org/linux/man-pages/man2/close.2.html>
- **[S5]** C99 draft, §7.19.3 — <https://busybox.net/~landley/c99-draft.html>
- **[S6]** Built-in Functions — `open()`, Python documentation — <https://docs.python.org/3/library/functions.html#open>
- **[S7]** Increase `io.DEFAULT_BUFFER_SIZE` — CPython issue 117151 — <https://github.com/python/cpython/issues/117151>

References · 2/27

- **[S9]** wininternl.h reference — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>
- **[S10]** NtTrace — Native API tracing for Windows — <https://rogerorr.github.io/NtTrace/>
- **[S11]** End-to-End Arguments in System Design — <https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf>
- **[S13]** fsync — The Open Group Base Specifications — <https://pubs.opengroup.org/onlinepubs/9699919799/functions/fsync.html>
- **[S14]** Predict-Observe-Explain — <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

References · 3/27

- **[S15]** The sad history of the Microsoft POSIX subsystem — <https://brianreiter.org/2010/08/24/the-sad-history-of-the-microsoft-posix-subsystem/>
- **[S17]** `io.IOBase.flush()` — Python documentation — <https://docs.python.org/3/library/io.html#io.IOBase.flush>
- **[S18]** Process Monitor — Sysinternals — <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>
- **[S19]** Inside Native Applications — <https://learn.microsoft.com/en-us/sysinternals/resources/inside-native-applications>
- **[S20]** `os` — Miscellaneous operating system interfaces, `os.write()`, Python documentation — <https://docs.python.org/3/library/os.html#os.write>

References · 4/27

- **[S21]** `fread(3)`, the page that documents `fwrite(3)` — Linux man-pages — <https://man7.org/linux/man-pages/man3/fwrite.3.html>
- **[S22]** Windows API index — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list>
- **[S31]** `ltrace(1)` — Linux man-pages — <https://man7.org/linux/man-pages/man1/ltrace.1.html>
- **[S32]** `ltrace` — project page — <https://ltrace.org/>
- **[S33]** NtTrace — source repository — <https://github.com/rogerorr/NtTrace>
- **[S34]** NtTrace64 capture of `cmd.exe /c echo Hello World!` — <https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/NtTrace64-CMD-ECHO-Output.txt>

References · 5/27

- **[S36]** NtTrace64 capture of iodemo.exe — <https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/iodemo-c/NtTrace64-IODEMO-Output.txt>
- **[S37]** iodemo on Linux and on Windows, the recorded runs (demo/iodemo-c/README.md) — <https://ondrasek.github.io/talks/2026-09-15-pb158-lecture-2-io-from-python-to-the-os/demo/iodemo-c/README.md>
- **[S38]** CloseHandle function (handleapi.h) — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/handleapi/nf-handleapi-closehandle>
- **[S39]** demo/close/README.md — the double-close capture on both platforms, 2026-09-20 — <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/close/README.md>
- **[S40]** FlushFileBuffers function (fileapi.h) — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-flushfilebuffers>

References · 6/27

- **[S41]** fsyncgate — the pgsql-hackers thread, March 2018 — <https://danluu.com/fsyncgate/>
- **[S42]** Can Applications Recover from fsync Failures? — USENIX ATC 2020 — <https://www.usenix.org/system/files/atc20-rebello.pdf>
- **[S43]** PRAGMA synchronous — SQLite documentation — https://www.sqlite.org/pragma.html#pragma_synchronous
- **[S44]** Delayed allocation and the zero-length file problem — Theodore Ts'o (2009) — <https://thunk.org/tytso/blog/2009/03/12/delayed-allocation-and-the-zero-length-file-problem/>
- **[S45]** Delayed allocation and the zero-length file problem — Valerie Aurora, LWN (2009) — <https://lwn.net/Articles/351422/>
- **[S46]** epoll(7) — Linux man-pages — <https://man7.org/linux/man-pages/man7/epoll.7.html>

References · 7/27

- **[S47]** select(2) — Linux man-pages — <https://man7.org/linux/man-pages/man2/select.2.html>
- **[S48]** The C10K problem — Dan Kegel (current revision) — <http://www.kegel.com/c10k.html>
- **[S49]** Comparing and Evaluating epoll, select, and poll Event Mechanisms — Gammo et al. (2004) — <https://www.kernel.org/doc/ols/2004/ols2004v1-pages-215-226.pdf>
- **[S50]** send(2) — Linux man-pages — <https://man7.org/linux/man-pages/man2/send.2.html>
- **[S51]** Classroom demonstrations: Learning tools or entertainment? — Crouch, Fagen, Callan and Mazur (2004) — <https://works.swarthmore.edu/fac-physics/203/>
- **[S52]** demo/durability/README.md — the writeback timeline and the four-mode counts, captured 2026-09-20 — <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/durability/README.md>

References · 8/27

- **[S53]** demo/waiting/README.md — the polling and epoll system-call counts, captured 2026-09-20 — <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/waiting/README.md>
- **[S54]** demo/iodemo-c/README.md — the pipe run and the traced netcat pair, captured 2026-09-20 — <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/iodemo-c/README.md>
- **[S55]** The C10K problem — Dan Kegel, Internet Archive snapshot of 8 May 1999 — <https://web.archive.org/web/19990508164301/http://www.kegel.com/c10k.html>
- **[S56]** demo/scatter/README.md — the writev trace and the sendfile counts, captured 2026-09-20 — <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/scatter/README.md>

References · 9/27

- **[S57]** demo/destinations-win32/README.md — the three-destination NtTrace capture, 2026-09-20 — <https://ondrasek.github.io/talks/2026-09-22-pb158-lecture-3-finishing-io-and-advanced-io/demo/destinations-win32/README.md>
- **[S58]** writev(2) — Linux man-pages — <https://man7.org/linux/man-pages/man2/writev.2.html>
- **[S59]** pipe(7) — Linux man-pages — <https://man7.org/linux/man-pages/man7/pipe.7.html>
- **[S60]** sendfile(2) — Linux man-pages — <https://man7.org/linux/man-pages/man2/sendfile.2.html>
- **[S61]** Module ngx_http_core_module — nginx documentation — https://nginx.org/en/docs/http/ngx_http_core_module.html#sendfile
- **[S62]** WriteFileGather function (fileapi.h) — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-writefilegather>

References · 10/27

- **[S63]** TransmitFile function (mswsock.h) — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/mswsock/nf-mswsock-transmitfile>
- **[S64]** socket — Low-level networking interface, Python documentation — <https://docs.python.org/3/library/socket.html#socket.socket.sendfile>
- **[S65]** valkey/src/networking.c — the reply path's writev — <https://github.com/valkey-io/valkey/blob/00a8a19554a39facb591837ea41ba4350260919d/src/networking.c>
- **[S66]** strace Wow Much Syscall — Gregg (2014). <https://www.brendangregg.com/blog/2014-05-11/strace-wow-much-syscall.html>
- **[S67]** fsync(2) — Linux man-pages — <https://man7.org/linux/man-pages/man2/fsync.2.html>

References · 11/27

- **[S68]** Linux kernel documentation — admin-guide/sysctl/vm.rst — <https://www.kernel.org/doc/html/latest/admin-guide/sysctl/vm.html>
- **[S69]** WSASend function (winsock2.h) — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/winsock2/nf-winsock2-wsasend>
- **[S70]** rename(2) — Linux man-pages — <https://man7.org/linux/man-pages/man2/rename.2.html>
- **[S71]** PostgreSQL commit 9ccdd7f — "PANIC on fsync() failure" (2018) — <https://git.postgresql.org/gitweb/?p=postgresql.git;a=commit;h=9ccdd7f66e3324d2b6d3dec282cfa9ff084083f1>
- **[S72]** Named Pipes — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/ipc/named-pipes>

References · 12/27

- **[S73]** PeekNamedPipe function (namedpipeapi.h) — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/namedpipeapi/nf-namedpipeapi-peeknamedpipe>
- **[S74]** GetQueuedCompletionStatus function (ioapiset.h) — Microsoft Learn — <https://learn.microsoft.com/en-us/windows/win32/api/ioapiset/nf-ioapiset-getqueuedcompletionstatus>
- **[S75]** io_uring(7) — Linux man-pages — https://man7.org/linux/man-pages/man7/io_uring.7.html
- **[V4]** A Successful write() Is a Handoff, Not a Save — The idiom the room already writes contains no durability step.
- **[V5]** A Successful write() Is a Handoff, Not a Save — The limit on the thesis. The bottom of the stack is not observable from software, and the deck says so.

References · 13/27

- **[V6]** Standard output is buffered differently depending on where it points — Removes the misconception that a Python write passes through the C library buffer.
- **[V8]** Windows and Linux place the stable syscall boundary at different layers — Takeaway 4, stated as a placement question rather than a quality judgement.
- **[V9]** Windows and Linux place the stable syscall boundary at different layers — Stops section 5 reading as a complaint about Windows.
- **[V10]** The System Call Boundary — Makes "a library call is not a system call" falsifiable with two tools instead of asserting it.
- **[V11]** The System Call Boundary — The closing honest-limits slide.

References · 14/27

- **[V14]** Where the Abstraction Leaks — Bounds section 3's vanishing-output demonstration so it does not teach the misconception it exists to remove.
- **[V17]** Observing a Running System — Why sections 1 and 2 need two runs.
- **[V20]** File IO and Durability — The device reports success before the bytes reach the medium, which is the sentence Lecture 2 stopped on.
- **[V21]** NtTrace — Native API tracing for Windows (Roger Orr) — Why this tool is usable on a student's own machine, and what its design bought.
- **[V22]** NtTrace — Native API tracing for Windows (Roger Orr) — The single string in the Windows trace that shows the layer difference.

References · 15/27

- **[V23]** NtTrace — Native API tracing for Windows (Roger Orr) — Comparison slide names the defect it has just cleared.
- **[V24]** PostgreSQL's fsync() surprise — Shipped storage engines answer the deferred-error problem by refusing to retry.
- **[V25]** Every Tracing Mechanism Trades Observation Cost for a Blind Spot — The honest-limits slide. The tool taught here is a teaching and debugging instrument, not an operational one.
- **[V28]** Sockets and Network I-O — One-slide bridge to networking.
- **[V29]** Sockets and Network I-O — The pipe row on the same slide.
- **[V30]** File IO and Durability — The row of the note's per-layer instrumentation table covering the kernel-to-device boundary.

References · 16/27

- **[V33]** File IO and Durability — The bottom arrow of the stack diagram, and the "emptied by" cell of level 4.
- **[V36]** Windows and Linux place the stable syscall boundary at different layers — The caveat on the outlier list.
- **[V37]** File IO and Durability — The Windows half of level 3's "emptied by" cell, and of the third arrow in the stack diagram.
- **[V38]** Windows and Linux place the stable syscall boundary at different layers — Why Linux nominates the layer it does, and why the other platforms need not.
- **[V39]** Every Tracing Mechanism Trades Observation Cost for a Blind Spot — What the lower-overhead mode named in [V25] actually buys, on a named workload.

References · 17/27

- **[V40]** PostgreSQL's fsync() surprise — How many storage engines abandoned the retry, and when.
- **[V41]** PostgreSQL's fsync() surprise — The independence of the three engines, which is what makes the never-retry slide an argument rather than an anecdote.
- **[V43]** Windows and Linux place the stable syscall boundary at different layers — The named list behind the outlier correction.
- **[V44]** File IO and Durability — Core claim, stated as a conjunction.
- **[V45]** File IO and Durability — The objection the room raises during section 3 — *but I can read the file back.*
- **[V46]** File IO and Durability — Windows half at equal depth.

References · 18/27

- **[V47]** File IO and Durability — The honest Windows answer is *undocumented*, which is a stronger and truer statement than *the same* or *safe*.
- **[V48]** File IO and Durability — Honesty slide. Why the 40 MiB file was written back sooner than the 4 MiB one, which is the first question the observation provokes.
- **[V49]** File IO and Durability — Close. A shipped database declines to trust the last software handoff and defends itself another way.
- **[V50]** Where the Abstraction Leaks — The thesis in one sentence, from the vault's own synthesis note.
- **[V51]** Where the Abstraction Leaks — Fixable half. The room needs the part software *can* get right, or the durability section reads as fatalism.

References · 19/27

- **[V52]** Where the Abstraction Leaks — Counter-position. Trusting the abstraction is respectable, and the note says exactly where it stops being respectable.
- **[V53]** A failed `fsync()` is not retryable — The kernel's *done* observed lying, in a trace of exactly the kind this course teaches the room to read.
- **[V54]** A failed `fsync()` is not retryable — Why the obvious repair is not a repair. It has the same shape as Lecture 2's slide "Never retry a failed close", one level down.
- **[V55]** A failed `fsync()` is not retryable — Windows slide. The asymmetry is about documentation, not about which platform is safer.
- **[V56]** PostgreSQL's `fsync()` surprise — The industry answer to an untrustworthy *done* is crash and replay, and it was never meant to last this long.

References · 20/27

- **[V57]** Readiness and Completion Models — Buffer ownership, not API taste, is what separates the two models, and it is why porting between them is a rewrite.
- **[V58]** Readiness and Completion Models — The sentence that stops the room reading *ready* as a guarantee, which is the misconception this model produces.
- **[V59]** Readiness and Completion Models — Correction to its own framing.
- **[V60]** Readiness and Completion Models — Windows slide. What the room should carry away as present tense, and what is only direction of travel.
- **[V61]** The C10K Problem Was About the Cost of Waiting, Not I-O — Predict slide. The controlled experiment behind the question, stated as what was held fixed and what was varied.

References · 21/27

- **[V62]** The C10K Problem Was About the Cost of Waiting, Not I-O — Explanation. epoll did not make a read faster, and the room needs that said before it draws the wrong conclusion from the counts.
- **[V63]** The C10K Problem Was About the Cost of Waiting, Not I-O — The page most often cited for this argument did not make it in 1999, so the deck cites the revision and the measurement instead.
- **[V64]** Sockets and Network I-O — Mechanism. Why read and write work on a socket, stated as a kernel structure rather than as a convention.
- **[V65]** Sockets and Network I-O — Boundary. The same interface does not mean the same contract, and the note names where the default stops.
- **[V66]** Sockets and Network I-O — Windows slide. The platforms agree on the mechanism and disagree on which layer they promise, which is the accurate statement.

References · 22/27

- **[V67]** Interprocess Communication — Boundary. The clean one-write-one-read result in the capture holds because fourteen bytes is small, and the room should know the threshold.
- **[V68]** io_uring and Windows IoRing — Completion on Both Platforms — Close. Where both platforms ended up, stated precisely enough not to read as "they converged".
- **[V69]** Blocking Is a Scheduler Decision, Not a Property of the I-O — The one question that orders polling, readiness and completion into a ladder rather than a feature list.
- **[V70]** Network Buffering and Delivery Semantics — Boundary. The capture shows one write and one read only because the payload is fourteen bytes, and the slide must say so.
- **[V71]** Standard output is buffered differently depending on where it points — Honest bound on its own demonstration.

References · 23/27

- **[V72]** Virtual Memory and the Page Cache — Handover. The page cache held the bytes for five seconds, and what that memory actually is belongs to the next lecture.
- **[V73]** io_uring and Windows IoRing — Completion on Both Platforms — Boundary. The demonstration uses pipes, and a student who tries the same loop on a file gets a mechanism that always answers yes.
- **[V74]** Readiness and Completion Models — What a completion model demands of the program, in Microsoft's own words, and the memory bill that follows from it.
- **[V75]** Readiness and Completion Models — The other half of the ownership comparison.
- **[V76]** Readiness and Completion Models — Why the choice of model is structural rather than a matter of API taste.
- **[V77]** Readiness and Completion Models — Takeaway 2 and section 4's definition slide.

References · 24/27

- **[V78]** File IO and Durability — What a dirty page is, what writeback is, and that the kernel rather than the program schedules it.
- **[V79]** io_uring and Windows IoRing — Completion on Both Platforms — Close. The version in which Linux acquired a completion interface.
- **[V80]** io_uring and Windows IoRing — Completion on Both Platforms — Close. The build in which Windows acquired a ring, paired with [V79].
- **[V81]** Readiness and Completion Models — Correction. The control case that shows readiness is the Unix family's answer rather than a Linux quirk.
- **[V82]** Interprocess Communication — Boundary. The threshold in bytes, and what happens above it, which the atomicity span [V67] does not state.

References · 25/27

- **[V83]** A failed `fsync()` is not retryable — Windows slide. The mechanism behind the undocumented contract, which is why no single answer exists.
- **[V84]** PostgreSQL's `fsync()` surprise — What InnoDB actually changed, which the count-and-year span [V40] does not carry.
- **[V85]** A failed `fsync()` is not retryable — Close. The log-and-replay answer, in PostgreSQL's own shipped documentation rather than as the deck's summary of it.
- **[V86]** File IO and Durability — The device lying is a reported practice rather than a theoretical possibility, and a shipped database says so.
- **[V87]** Sockets and Network I-O — Boundary. A concrete operation that does not transfer, which the strong-default span [V65] asserts without instantiating.

References · 26/27

- **[V88]** `io_uring` and Windows `IoRing` — Completion on Both Platforms — Why the regular-file gap mattered enough to build a second model, which the `epoll-and-regular-files` span [V73] does not say.
- **[V89]** A failed `fsync()` is not retryable — The mechanism behind the failed retry, which [V54]'s one-sentence span states without explaining.
- **[V90]** A failed `fsync()` is not retryable — How long the misreading stood before anyone noticed, which is what makes it a contract failure rather than a careless bug.
- **[V91]** File IO and Durability — Windows slide. The judgement itself, from the note that makes it, rather than asserted by the deck on the strength of a missing wiki row.
- **[V92]** Readiness and Completion Models — The `select(2)` BUGS caveat applies to the interface the demonstration actually used.

References · 27/27

- **[V93]** PostgreSQL's fsync() surprise — What PostgreSQL actually changed, and that the change reached every release rather than only the current one.
- **[V94]** The C10K Problem Was About the Cost of Waiting, Not I-O — Kegel's arithmetic for why a thread per connection ran out of virtual memory.
- **[V95]** The C10K Problem Was About the Cost of Waiting, Not I-O — Banga, Mogul and Druschel on the scheduler half of the same bill.
- **[V96]** The C10K Problem Was About the Cost of Waiting, Not I-O — What unifies the thread rung with the polling rung: a charge levied per connection, not per byte.

The handout PDF carries the full reference list, and a one-page summary of every [V] source with its own external references.

TAKE IT WITH YOU

Everything from this talk

[Slides \(PDF\)](#)

[Handout \(PDF\)](#)

[Speaker notes \(PDF\)](#)

[Everything \(.zip\) — readable by an AI agent](#)

ondrasek.github.io/talks

me@ondrejkrasicek.com